

# Microservice Architecture

## Знакомство с Kubernetes, основные понятия и архитектура

otus.ru




**Проверить, идет ли запись**

# Меня хорошо видно & слышно?



Ставим “+”, если все хорошо
“-”, если есть проблемы



# Майоров Антон

Старший разработчик ПО

- 10+ лет опыта разработки ПО на платформе .NET
- Специализация: приложения со сложной бизнес логикой
- Последние 4 года - старший системный разработчик в Ренессанс Страхование

@amaiorov

# Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе
#**канал группы**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

# Маршрут вебинара

- Знакомство
- Архитектура Kubernetes
- Инструменты взаимодействия с кластером
- Основные объекты k8s
- Демо
- Рефлексия

# Цели вебинара

## К концу занятия вы сможете

1. Разобраться с архитектурой кластера
2. Взаимодействовать с кластером
3. Понимать базовые сущности k8s

# Архитектура Kubernetes

# Kubernetes

**Kubernetes** –это программная система, которая позволяет легко разворачивать контейнеризированные приложения и управлять ими

Она использует возможности контейнеров Linux для запуска разнородных приложений без необходимости знать какие-либо внутренние детали этих приложений и без необходимости вручную разворачивать эти приложения на каждом хосте

Kubernetes позволяет выполнять ваши программные приложения на тысячах компьютерных узлов, как если бы все эти узлы были одним огромным компьютером

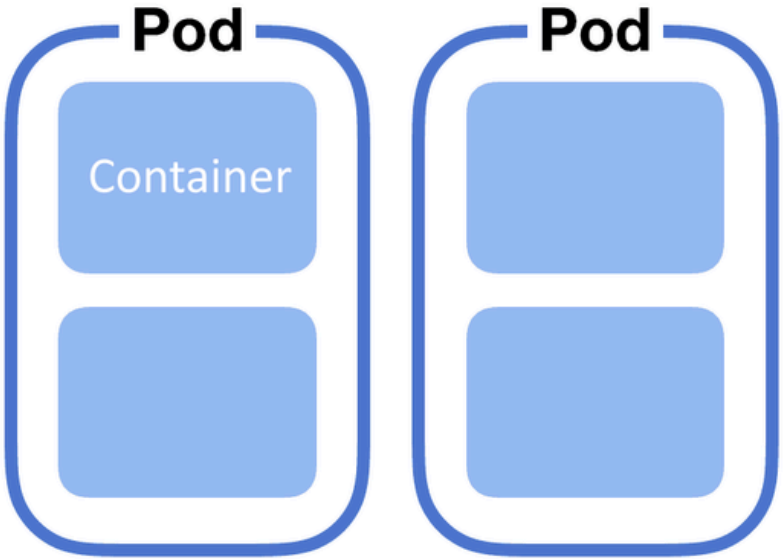
На текущий момент является самым популярным, самым развиваемым opensource проектом. На его основе создаются коммерческие продукты: openshift, tanzu, rancher и т.д, и managed кластера кубернетеса.

Nomad, mesos, dc/os, dockerswarm–развиваются похуже и зачастую уступают.

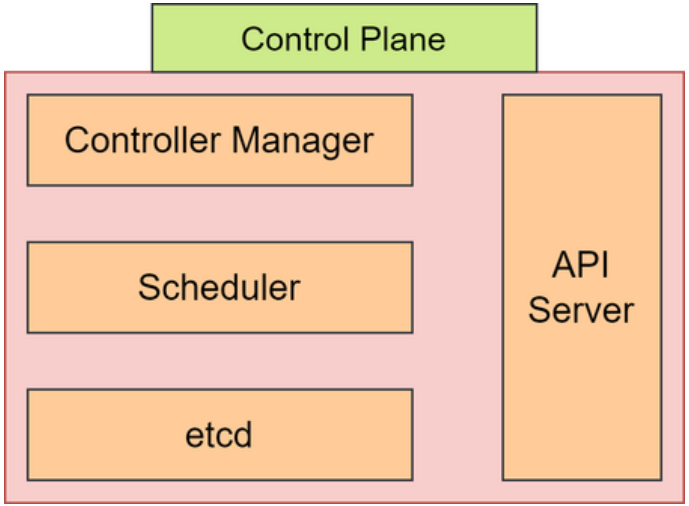
# Pod



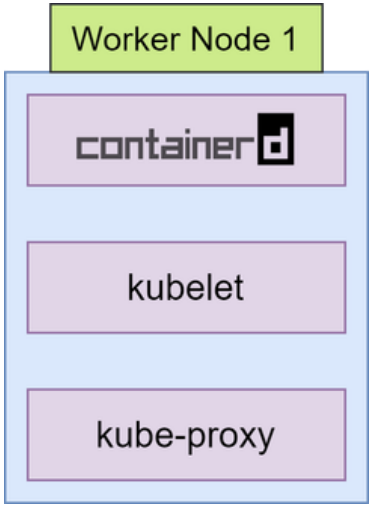

**Pod** - абстракция k8s, соответствующая контейнеру. Атомарная единица рабочей нагрузки в k8s. Одному поду может соответствовать несколько контейнеров (но обычно это 1 контейнер).



# Control Plane & Worker Nodes

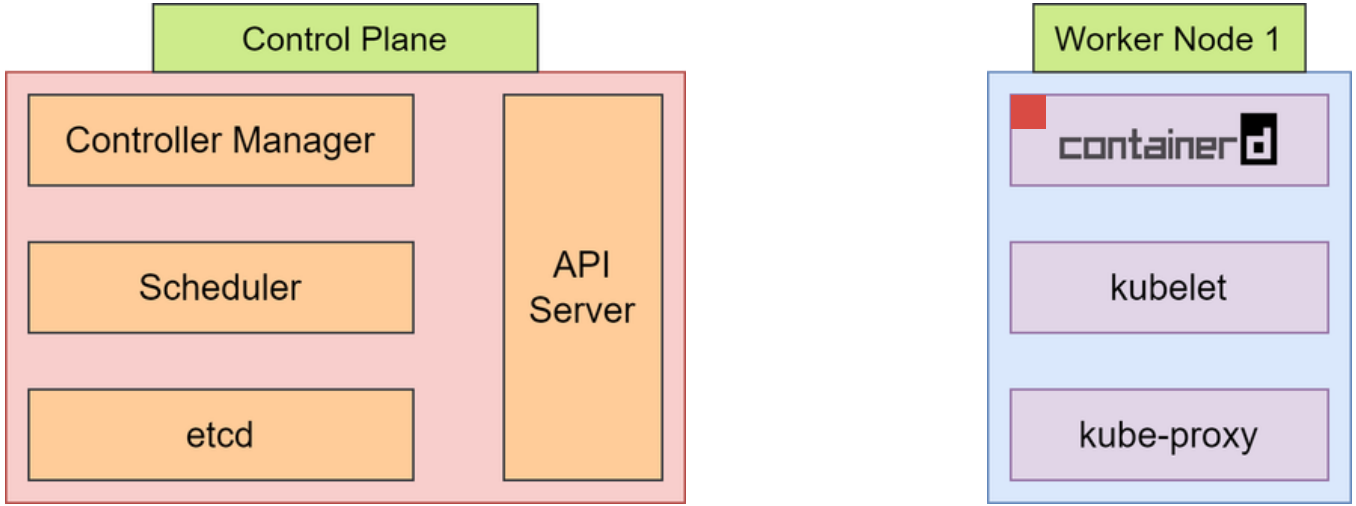



управляет состоянием всего кластера



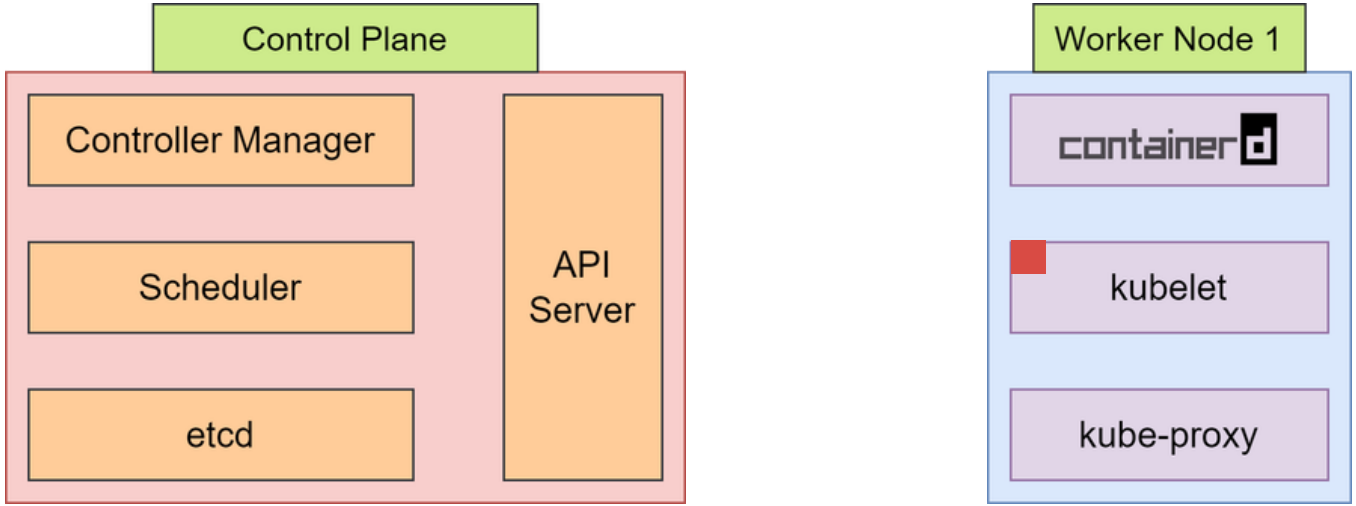

выполняет ваше приложение

# Container Runtime



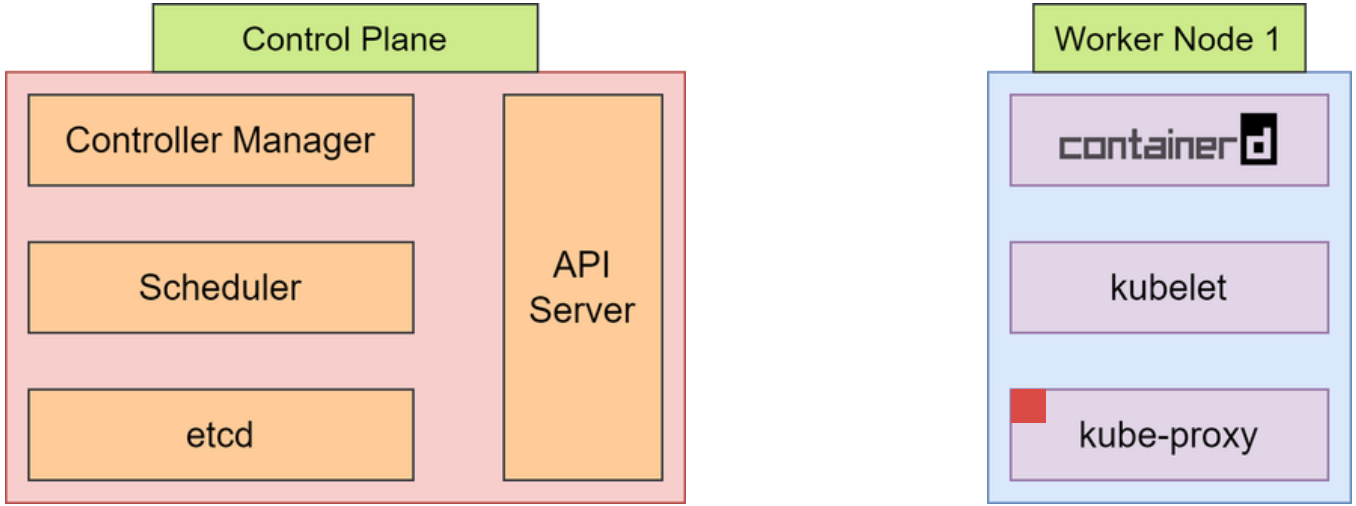

**Container runtime** - среда, позволяющая запускать контейнеры в рабочем узле. Самый распространенный вариант - container-d

# kubelet



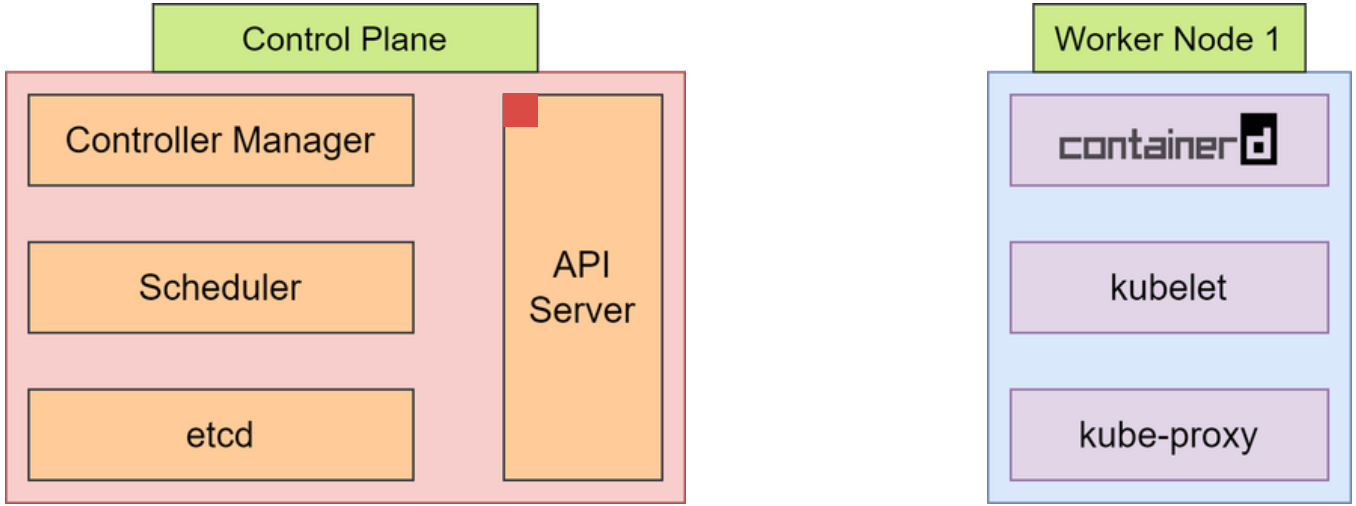

**kubelet** - инициирует запуск и остановку контейнеров, выделяя им CPU, память и другие ресурсы.

# kube - proxy



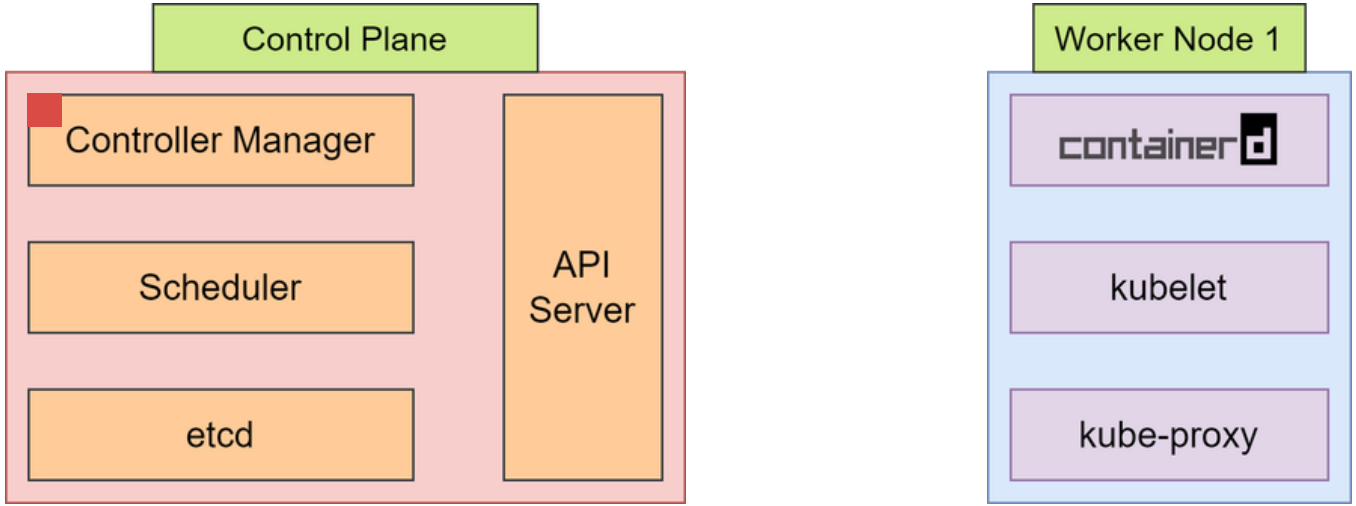

**kube - proxy** - участвует в маршрутизации трафика к подам

# API - Server



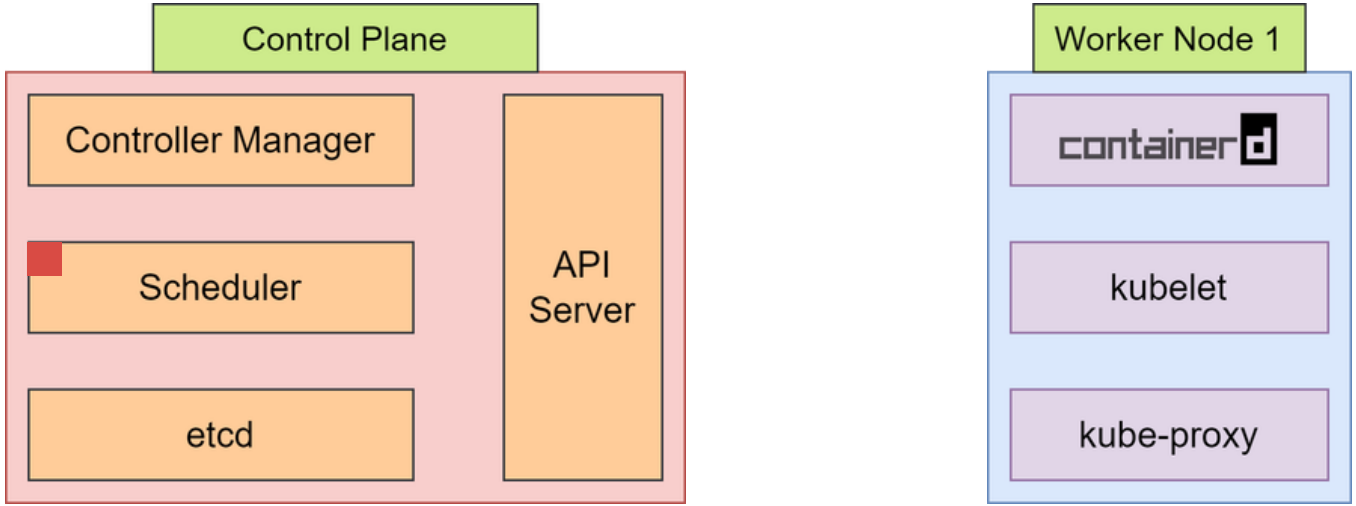

**API - Server** - точка входа для внешних и внутренних команд / запросов: через него работают kubectl, UI и другие клиенты.

# Controller Manager



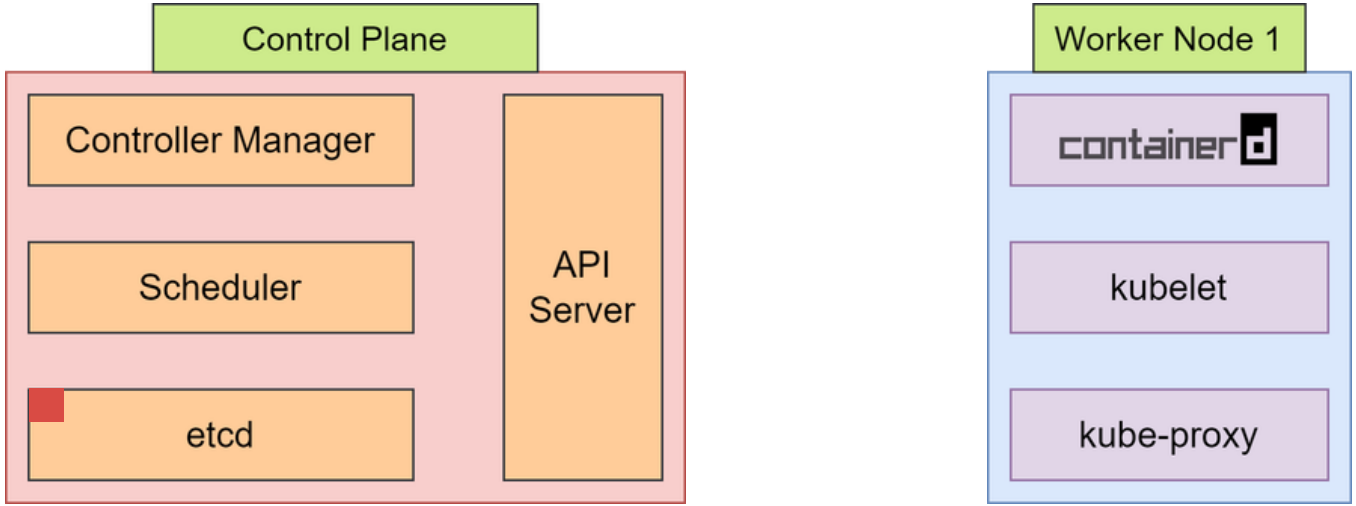

**Controller Manager** - постоянно сверяет фактическое состояние с желаемым. Если Pod исчез, контроллер инициирует его восстановление.

# Scheduler



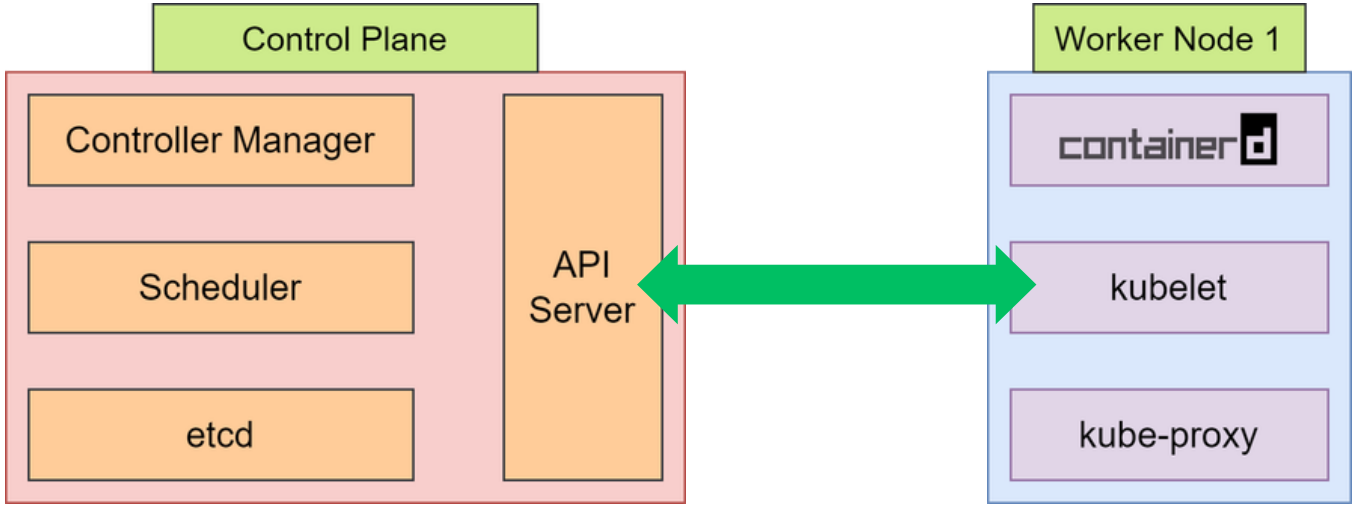

**Scheduler** - выбирает worker-node для нового Pod’a, учитывая доступные ресурсы и ограничения размещения.

# etcd



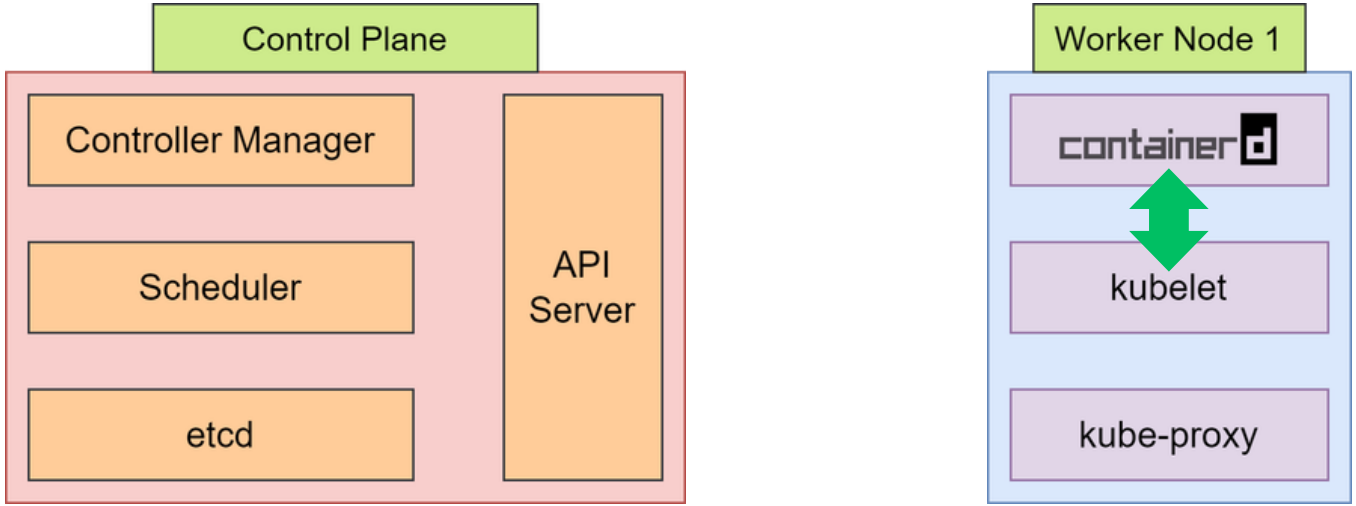

**etcd** - распределённое key-value хранилище состояния кластера: какие ресурсы существуют, какой у них текущий статус.

# kubelet - API Server



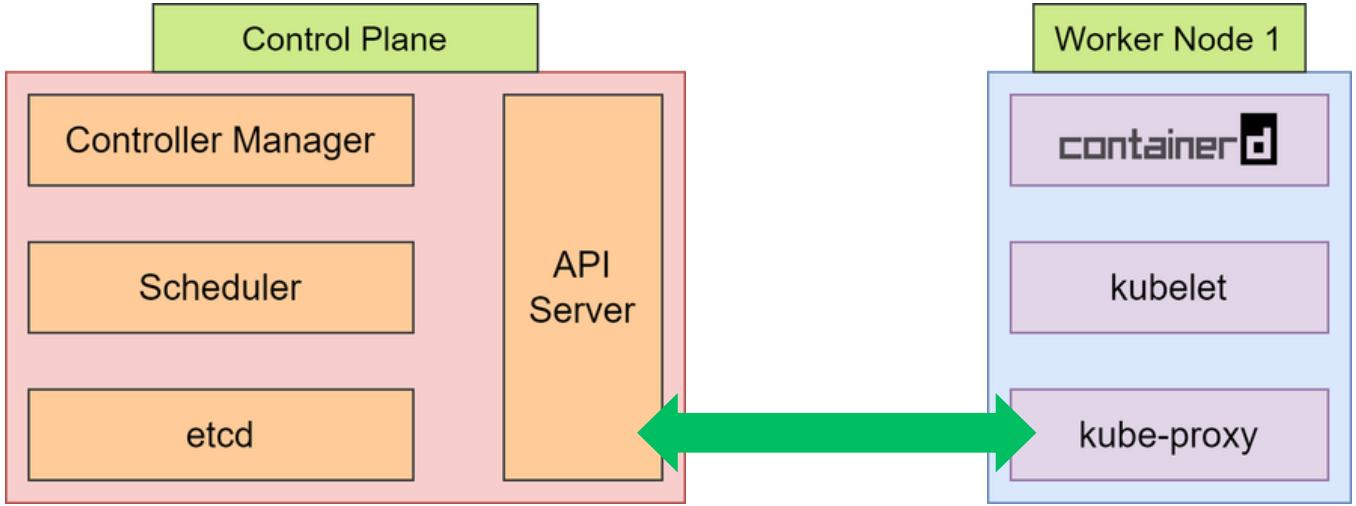

**kubelet** запрашивает информацию о подах, которые должны быть запущены на данной ноде в **API Server** и сообщает обратно информацию о запущенных подах

# Container Runtime - kubelet



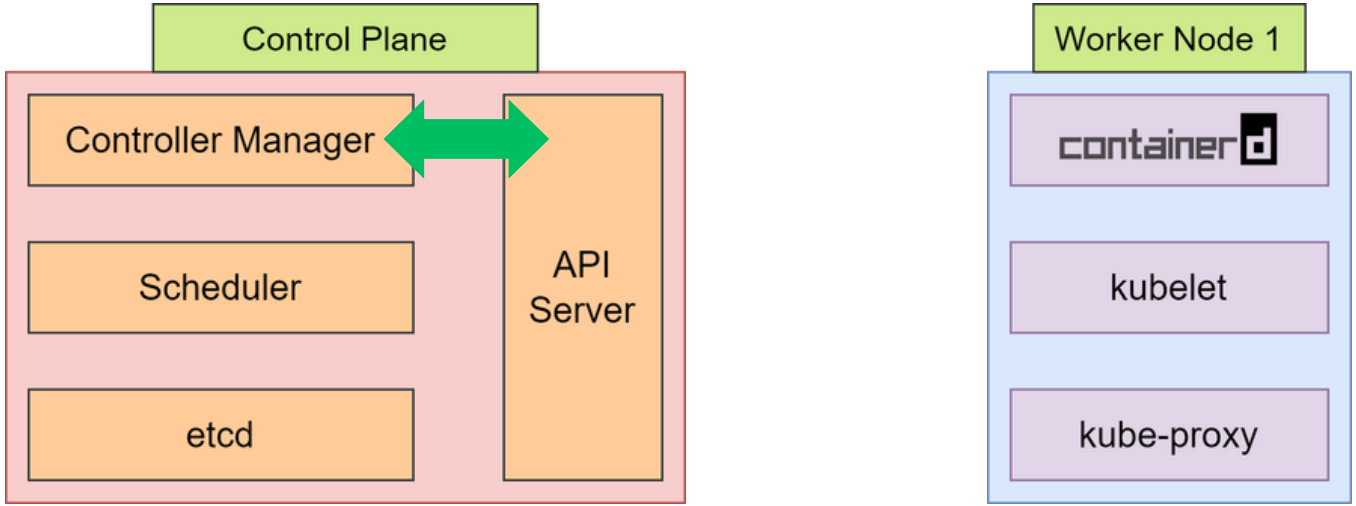

**kubelet** инициирует запуск и остановку контейнеров в **Container Runtime**.
**Container Runtime** сообщает **kubelet** о состоянии контейнеров

# kube-proxy - API Server



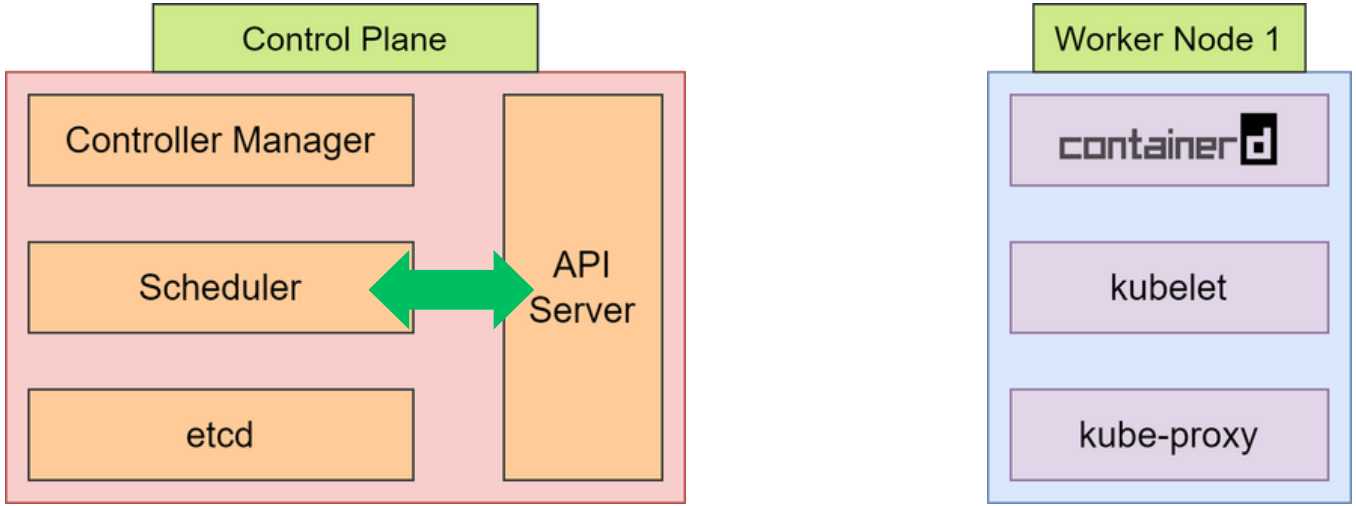

**kube-proxy** запрашивает в **API Server** информацию о правилах сопоставления виртуального адреса Service и реального адреса подов. Обеспечивает маршрутизацию трафика к подам

# Controller Manager - API Server



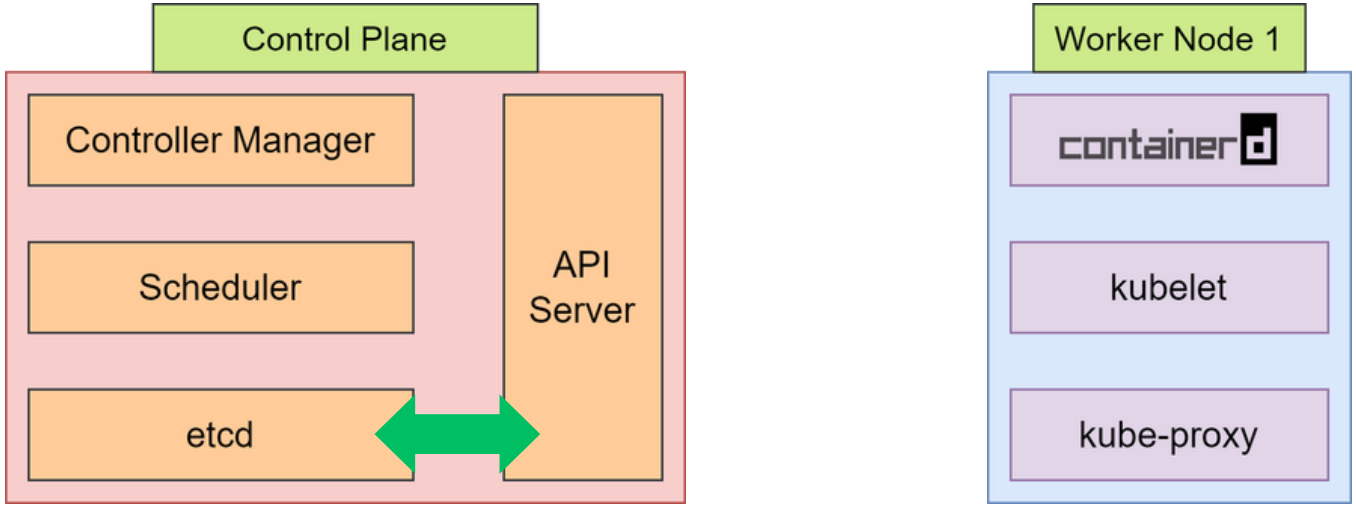

**Controller Manager** отслеживает состояние подов и инициирует создание  нового в случае падения через **API Server**

# Scheduler - API Server



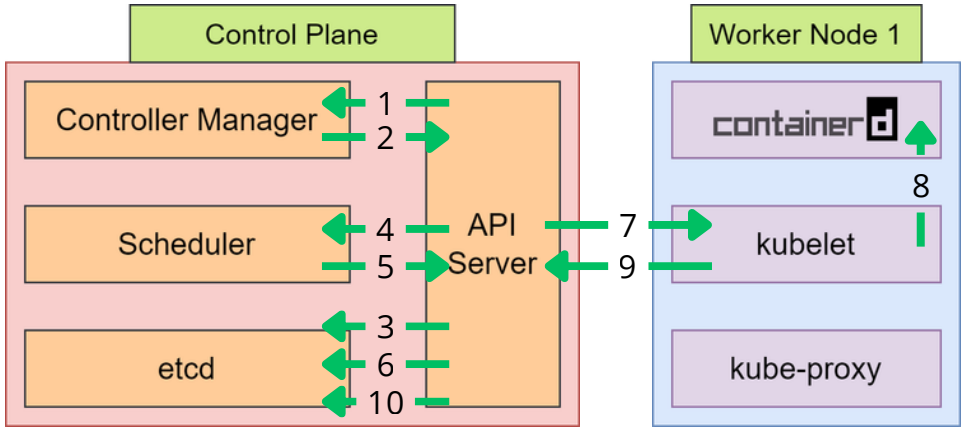

**Scheduler** планирует размещение пода на нектором Worker Node через **API Server**

# API Server - etcd




**etcd** хранит всю информацию о кластере, которую сообщает **API Server**

# Восстановление пода при отказе ноды




1. пода нет, число реплик меньше заданного
2. создай под (пока без указания ноды)
3. сохраняю информацию о поде в etcd
4. есть под без ноды
5. назначаю поду ноду 1
6. сохраняю информацию о поде в etcd
7. есть новый под для ноды 1
8. запусти контейнер для нового пода
9. новый под запущен и готов
10. сохраняю информацию о поде в etcd

# Вопросы?



Ставим “+”,
если вопросы есть



Ставим “–”,
если вопросов нет

# Внешнее управление кластером

# Kubectl

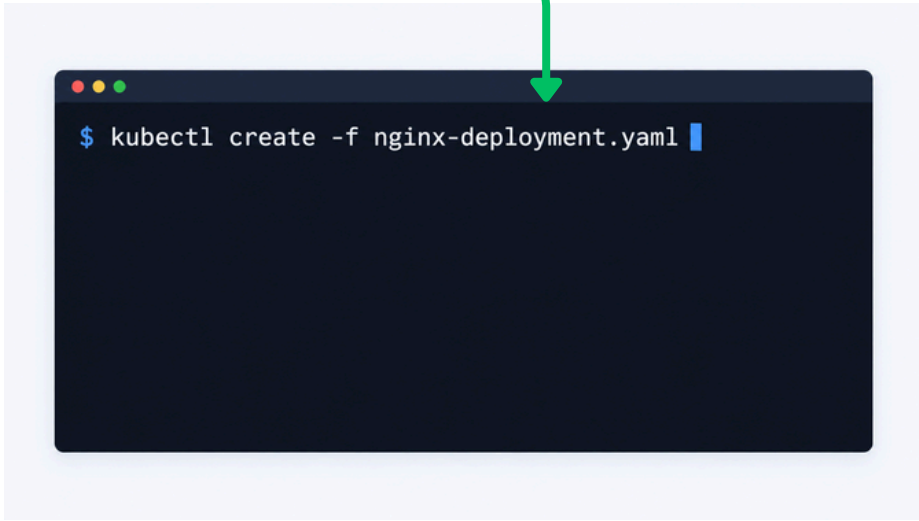
**Kubectl** – утилита коммандной строки, позволяющая отправлять запросы к кластеру




Инструкция по установке: https://kubernetes.io/docs/tasks/tools/

# YAML Manifests

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-nginx
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```



# Императивный vs Декларативный




1. Удобен при тестировании.
2. Подходит для быстрых разовых задач.




1. Сохраняется история изменений конфигурации.
2. **Инфраструктуру можно описывать как код** и хранить в Git-репозитории.
3. Возможны совместная работа и проверка изменений.
4. Изменения становятся более прозрачными.

# Вопросы?



Ставим “+”,
если вопросы есть

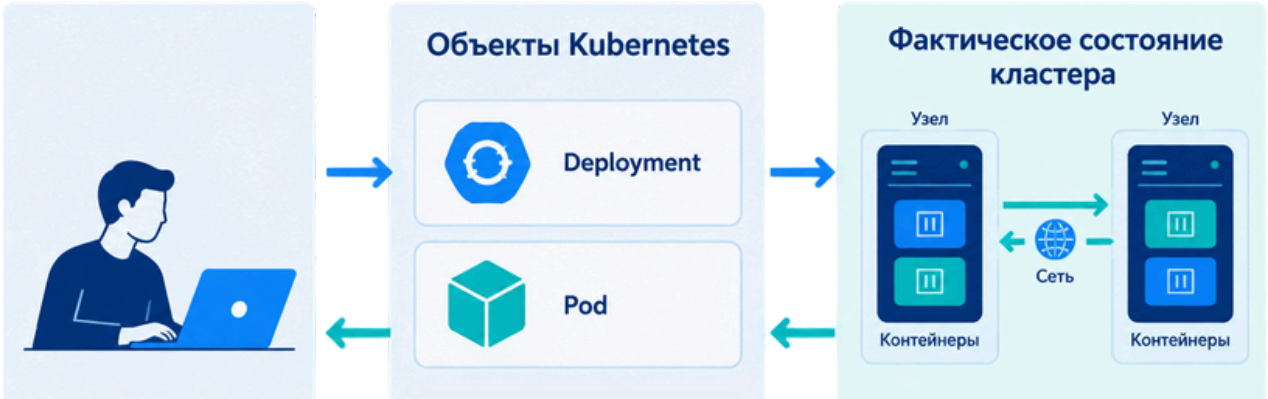


Ставим “–”,
если вопросов нет

# Объекты k8s и их описание

# Объекты k8s

Объекты в Kubernetes –это хранящиеся внутри Kubernetes сущности (ресурсы). И Kubernetes их использует для представления состояния кластера.

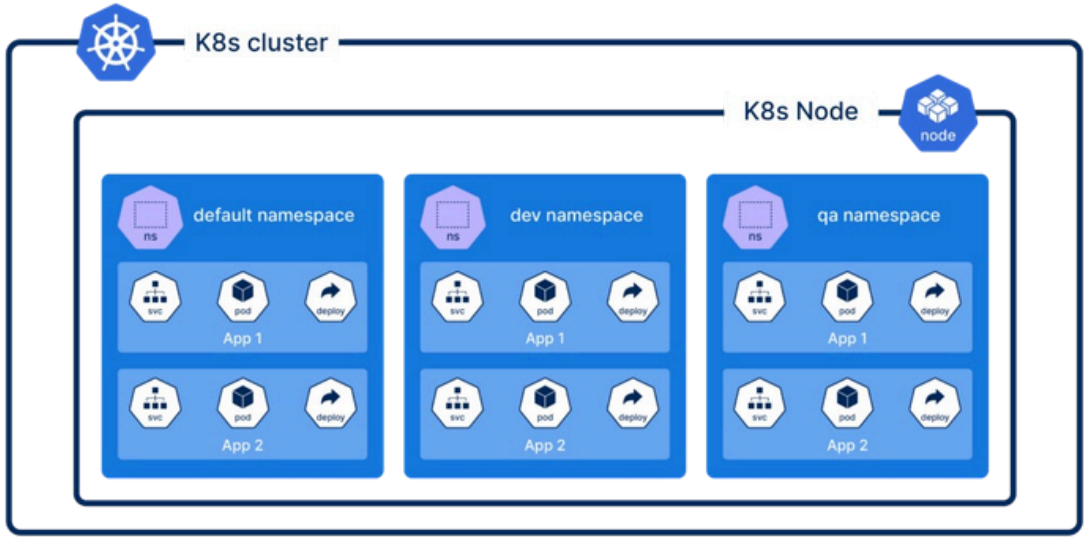



- Увеличить количество инстансов
- Полностью потушить деплой
- Убрать сервис из балансировки

- Какие приложения сейчас запущены?
- Какие ресурсы доступны?
- Политики поведения (рестарты, обновления)

# Пространства имен

**Namespace** - способ изоляции объектов k8s внутри одного кластера




Имена объектов должны быть уникальны в рамках одного namespace

Можно задавать права доступа, квоты, лимиты и т.д в рамках namespace

# Pod. Пример конфигурации

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

| Поле | Зачем |
| --- | --- |
| kind | Тип объекта - Pod, Deployment и т.д. |
| apiVersion | Сообщает k8s, по какой версии API читать описание объекта. Вместе с kind оно определяет, какой ресурс описывается и какая схема полей к нему применяется. |
| metadata | Имя, метки, аннотации |
| spec | Желаемое состояние объекта |

# Pod. Пробы

```yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-exec
spec:
  containers:
  - name: liveness
    image: registry.k8s.io/busybox
    args:
    - /bin/sh
    - -c
    - touch /tmp/healthy; sleep 30; rm -f /
    livenessProbe:
      exec:
        command:
        - cat
        - /tmp/healthy
      initialDelaySeconds: 5
      periodSeconds: 5
```

| Вид | Зачем | Действие |
|---|---|---|
| startup | Приложение уже завершило запуск? | Пока проверка не успешна, readinessProbe и livenessProbe не выполняются. Если допустимое число неудач исчерпано, контейнер останавливается и может быть перезапущен. |
| readiness | Приложение ещё работает, или оно зависло? | Pod перестаёт быть Ready и исключается из доступных endpoints подходящих Service. |
| liveness | Можно ли сейчас отправлять приложению запросы? | kubelet останавливает и перезапускает контейнер согласно политике перезапуска Pod |

Механизмы: http, grpc, tcp, exec

# Pod. Volumes

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-webserver
spec:
  os: { name: linux }
  nodeSelector:
    kubernetes.io/os: linux
  containers:
  - name: test-webserver
    image: registry.k8s.io/test-webserver:latest
    volumeMounts:
    - mountPath: /var/local/aaa
      name: mydir
    - mountPath: /var/local/aaa/1.txt
      name: myfile
  volumes:
  - name: mydir
    hostPath:
      # Ensure the file directory is created.
      path: /var/local/aaa
      type: DirectoryOrCreate
  - name: myfile
    hostPath:
      path: /var/local/aaa/1.txt
      type: FileOrCreate
```

| Тип в spec.volumes | Откуда берутся данные | Типичное применение |
|---|---|---|
| emptyDir | Пустое пространство, создаваемое для Pod | Временные файлы, обмен данными между контейнерами одного Pod |
| **hostPath** | Путь в файловой системе узла | Доступ к файлу или каталогу на конкретном узле |
| configMap / secret | Объекты Kubernetes | Конфигурация и секреты, доступные контейнеру как файлы |
| persistentVolumeClaim | Хранилище, полученное через PVC и PV | Данные, которые должны жить независимо от конкретного Pod |
| nfs | Сетевая файловая система NFS | Доступ к существующему сетевому каталогу |

# Вопросы?



Ставим “+”,
если вопросы есть



Ставим “–”,
если вопросов нет

# Итоги

# Вопросы для повторения

1. Из каких компонентов состоит Worker Node и какая у них функция?
2. Из каких компонентов состоит Control Plane и какая у них функция?
3. Как восстанавливается контейнер при падении ноды?
4. Какие есть способы взаимодействия с кластером и когда какой нужно применять?
5. Каков общий набор полей при описании объекта k8s?
6. Какие бывают пробы и зачем они нужны?
7. Какие бывают volumes?

# Заполните, пожалуйста, опрос о занятии по ссылке в чате